AGENT 1 - MONITORING
Démarche de travail

Du banc d'essai à la détection d'anomalies, et la suite vers un agent intelligent

Projet : Intelligent Multi-Agent System for Autonomous DevOps / MLOps Pipelines

Élément	Valeur
Responsable	Member 1 - Farah Marzougui
Composant	Agent 1 - Monitoring & Anomaly Detection (src/agents/monitoring/)
État	Étapes 0 à 6 réalisées ; 124 tests automatiques réussis
Date	9 octobre 2026

1. Objectif et rôle de l'agent

L'Agent 1 observe les services, détecte les comportements anormaux et vérifiera la récupération après une réparation. Il rapporte des **faits observés**, jamais la cause : le *pourquoi* appartient à l'Agent Diagnostic.

Mon agent répond à	L'Agent Diagnostic répond à
Quoi, où, quand, à quel point c'est grave	Pourquoi, quelle cause probable
« Mémoire +30 MB/min depuis 6 min »	« C'est une fuite mémoire »
« 340 erreurs CONNECTION_REFUSED »	« La base de données est tombée »

2. La démarche suivie

Quatre principes ont guidé tout le travail :

Principe	Ce que cela veut dire	Pourquoi
1. Un banc d'essai d'abord	Un faux service (payment-service) qu'on rend malade volontairement	On connaît la panne à l'avance, donc on peut vérifier si l'agent dit la vérité
2. Prouver chaque étape	Tests automatiques + test réel avec une panne injectée	Avancer sur une base fiable, sans erreur cachée
3. Fiabilité avant sophistication	Règles et statistiques explicables ; LLM seulement pour le texte	Une alerte doit être fiable, reproductible et explicable
4. Développement incrémental	Une couche à la fois, chacune utilisable et démontrable	Toujours une version qui fonctionne, même si le temps manque

La méthode de test

On injecte une panne **connue** (par exemple +30 MB/min), on demande à l'agent ce qu'il voit, puis on compare. C'est comme tester un thermomètre dans de l'eau à exactement 30 °C : ce n'est pas le service qu'on teste, c'est l'agent.

3. Travail réalisé, étape par étape

Étape 0 - Préparer l'environnement et comprendre les contrats

But : connaître le format exact des messages que l'agent échange avec les autres.

Ce qui a été fait : environnement Python 3.13 (.venv), lecture des contrats (AnomalyDetected, RecoveryVerified). Le contrat refuse tout champ inconnu.

Fichiers : `src/contracts/payloads.py, src/agents/monitoring/ports.py`

Résultat vérifié : les 47 tests d'origine du projet passent.

Étape 1 - Le banc d'essai (le « patient »)

But : disposer de pannes connues pour tester l'agent.

Ce qui a été fait : application payment-service qui publie ses chiffres sur /metrics ; boutons de panne /chaos/* (mémoire, erreurs, latence, CPU, reset) actifs seulement en local ; Prometheus et cAdvisor dans Docker ; générateur de trafic ; journal horodaté des pannes.

Fichiers : `experiments/target_app/, experiments/load/traffic.py, compose.yaml`

Résultat vérifié : panne mémoire injectée à 60 MB/min, mesurée à **59,8 MB/min**.

Étape 2 - Le collecteur Prometheus (les « yeux »)

But : permettre à l'agent de lire seul les chiffres.

Ce qui a été fait : 6 signaux lus en parallèle (mémoire, pente, CPU, trafic, erreurs, latence p95) ; protection par timeout, nouvelles tentatives et disjoncteur ; « inconnu » vaut None, jamais zéro.

Fichiers : `collectors/prometheus.py, collectors/http.py, src/platform/resilience.py`

Résultat vérifié : 30 MB/min et 20 % injectés, lus **30,1 MB/min et 20,3 %** ; Prometheus arrêté : pas de crash.

Étape 3 - La détection SLO (la « limite fixe »)

But : savoir si les utilisateurs sont vraiment touchés.

Ce qui a été fait : deux promesses de qualité (99,5 % de visites sans erreur, 99,5 % en 500 ms ou moins) ; burn rate calculé sur deux durées (méthode du SRE Workbook de Google).

Fichiers : `detectors/slo.py`

Résultat vérifié : 20 % d'erreurs : alerte **HIGH** (burn rate environ 48) ; aucune alerte au repos.

Étape 4 - La détection statistique (les « signes avant-coureurs »)

But : voir les problèmes avant que les utilisateurs souffrent.

Ce qui a été fait : comparaison avec l'habitude du service : croissance (pente + R^2) et changement de niveau (médiane + z robuste) ; un pic isolé ne déclenche jamais d'alerte.

Fichiers : `detectors/statistical.py`

Résultat vérifié : mémoire détectée en **2 min 48 s** (plus de 12 min avant le crash) ; latence de 487 ms, invisible pour le SLO, en **2 min 33 s** ; aucune fausse alerte.

Étape 5 - L'analyse des logs (le « genre » de problème)

But : savoir quel type d'erreur apparaît.

Ce qui a été fait : secrets masqués en premier ; Drain3 regroupe les lignes en modèles ; étiquettes dans une liste fermée (règles d'abord, Gemini en option pour les cas inconnus) ; détection des modèles nouveaux ou en forte hausse ; ajout de Loki et Promtail.

Fichiers : `detectors/logs.py, collectors/loki.py, labelers/gemini.py`

Résultat vérifié : 23 tests automatiques réussis ; **test réel avec Loki pas encore fait**.

Étape 6 - Le regroupement en incidents

But : une seule alerte par problème.

Ce qui a été fait : signature service + environnement + type de symptôme ; mise à jour d'un incident ouvert au lieu d'en créer un nouveau ; refroidissement de 15 min ; notification seulement si quelque chose change.

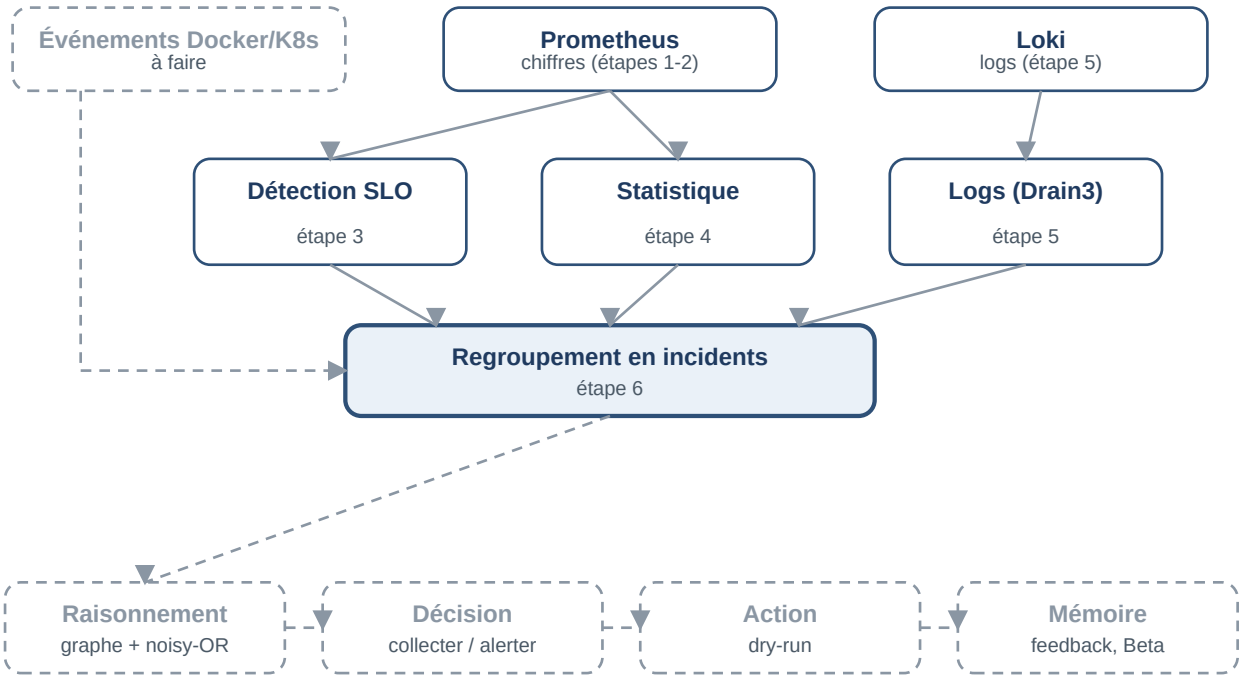
Fichiers : `grouping.py`

Résultat vérifié : 40 observations d'une même panne donnent **1 incident et 1 seule notification**.

4. Architecture actuelle

L'agent lit deux sources, passe les données dans trois détecteurs indépendants, puis regroupe leurs symptômes en incidents. Le bas du schéma est la suite du travail.

Trait plein : fait Pointillé : à faire



5. Choix techniques

Choix	Raison
Statistiques plutôt qu'un LLM pour les chiffres	Fiables, reproductibles, explicables, sans données d'entraînement
Burn rate sur deux durées	Alerte seulement si le problème est important et toujours en cours
Médiane plutôt que moyenne	Un pic isolé ne fausse pas le résultat
« Inconnu » n'est jamais « normal »	Sans données, l'agent dit INSUFFICIENT_DATA au lieu de dire que tout va bien
Gemini en option, uniquement pour les logs	Le texte est son point fort ; secrets masqués, liste fermée, l'agent marche sans lui
Paramètres en configuration	Les seuils se changent sans toucher la logique
Interface commune Docker / Kubernetes	Kubernetes est la cible à long terme ; Docker sert au banc d'essai

6. Limites actuelles (état honnête)

- L'agent n'est **pas encore complet** : il perçoit et décide, mais ne fonctionne pas encore en continu et ne publie pas encore ses incidents.
- Le test réel de l'étape 5 (Loki) reste à faire.
- Les incidents ouverts sont gardés en mémoire ; un stockage Redis est nécessaire pour survivre à un redémarrage.
- Le collecteur d'événements (OOMKilled, redémarrages, déploiements) n'existe pas encore.
- Les seuils sont des valeurs de départ, à calibrer pendant l'évaluation.

- Le travail n'est pas encore enregistré dans Git.

7. La suite : vers un agent intelligent

L'objectif est de passer de « je détecte » à la boucle complète d'un agent : **Perception, Raisonnement, Décision, Action, Mémoire**. La démarche reste incrémentale : la plus grande valeur d'abord, le plus risqué en dernier.

Ordre	Étape	Ce que l'agent saura faire	Démonstration possible
1	Boucle autonome	Surveiller seul, toutes les 1 à 5 min	« J'injecte une panne, l'agent ouvre l'incident tout seul »
2	Raisonnement par graphe + noisy-OR	Classer les services suspects grâce au graphe de dépendances	« La base de données est la cause la plus probable » (avec sa probabilité)
3	Décision « collecter plus »	Chercher des preuves quand il hésite	« Il interroge d'autres métriques avant d'agir »
4	Mémoire : feedback (loi Beta)	Apprendre des vraies et fausses alertes	« Il ajuste sa confiance après ma correction »
5	Chronos-Bolt en comparaison	Prévoir une fourchette normale	Tableau comparatif avec la méthode actuelle
6	Actions risquées + Thompson sampling	Proposer une action, en dry-run	« Il propose un redémarrage sans l'exécuter »

Pourquoi cet ordre

Chaque étape est utilisable et démontrable seule. On commence par ce qui apporte le plus pour le moins d'effort (l'autonomie), et on finit par ce qui est le plus risqué (agir sur le système) et qui demande le plus de données (apprendre le meilleur choix d'action).

Deux conditions avant de commencer

1. Valider avec l'encadrant le raisonnement sur la cause, proche du rôle de l'Agent Diagnostic (il peut être présenté comme des hypothèses classées).
2. Enregistrer le travail actuel dans Git.

Sur l'intelligence de l'agent

Aucun modèle n'est entraîné et aucun LLM n'est utilisé pour les chiffres. Chronos-Bolt est un modèle pré-entraîné, utilisé sans entraînement et seulement en comparaison. L'intelligence viendra surtout du **raisonnement** sur le graphe de dépendances, de la **décision** et de la **mémoire**.

8. Glossaire

Terme	Signification simple
Prometheus	Base qui enregistre les chiffres des services dans le temps
Loki	Base qui enregistre les logs (les phrases écrites par les services)
SLO	Promesse de qualité chiffrée, par exemple 99,5 % de visites réussies
Budget d'erreur	La part d'erreurs autorisée par la promesse (ici 0,5 %)
Burn rate	Vitesse à laquelle le budget d'erreur est consommé
Médiane, MAD, z robuste	Mesures de « normal » et d'écart, insensibles aux pics isolés
Drain3	Algorithme qui regroupe les phrases de logs qui se ressemblent
LLM (Gemini)	IA qui comprend le texte, utilisée seulement pour étiqueter les logs inconnus
Noisy-OR	Calcul de probabilité qui combine plusieurs indices

Terme	Signification simple
Dry-run	Mode simulation : l'agent dit ce qu'il ferait, sans le faire